Les namespaces

En PHP, les **namespaces** (ou espaces de noms) résolvent le problème des conflits de noms de classes, de fonctions ou de constantes dans une application. Ils fonctionnent de manière analogue aux **dossiers de votre système de fichiers** : deux fichiers distincts peuvent porter le même nom à condition d'être placés dans des dossiers différents.

1. Pourquoi utiliser les namespaces ?

Sans namespaces, si vous installez une bibliothèque externe qui possède une classe `User`, et que votre propre application possède déjà une classe `User`, PHP génère une erreur fatale.

- **Éviter les collisions de noms** entre votre code et les dépendances tierces.
- **Raccourcir le code** en évitant l'ancienne pratique qui consistait à nommer les classes avec de longs préfixes (ex: `Zend_Search_Lucene_Analysis_Analyzer`).
- **Améliorer l'organisation** logique de l'application.

2. Déclarer un namespace

La déclaration d'un namespace doit impérativement être la **toute première instruction** du fichier PHP, placée juste après la balise `<?php`. On utilise le caractère antislash `\` pour symboliser les sous-dossiers.

```
<?php
namespace App\Services; // Déclaration de l'espace de nom

class Mailer {
    public function send() {
        return "Email envoyé !";
    }
}
```

3. Utiliser une classe d'un autre namespace

Pour appeler une classe située dans un autre espace de noms, PHP propose trois approches principales :

A. Le chemin absolu (Fully Qualified Name)

Vous spécifiez le chemin complet en partant de la racine avec un antislash initial `\`.

```
$mailer = new \App\Services\Mailer();
```

B. L'importation avec le mot-clé `use`

C'est la méthode recommandée. Elle permet d'importer la classe au début de votre fichier pour pouvoir l'appeler directement par son nom.

```
<?php
namespace App\Controllers;

use App\Services\Mailer; // Importation de la classe

class UserController {
    public function register() {
        $mailer = new Mailer(); // Appel direct simplifié
    }
}
```

C. L'utilisation d'alias avec `as`

Si deux classes portent le même nom mais viennent de namespaces différents, vous pouvez renommer l'une d'elles à la volée pour éviter un conflit.

```
use App\Services\Mailer;
use ExternalLib\Mailer as ExternalMailer; // Création d'un alias

$local = new Mailer();
$external = new ExternalMailer();
```

4. Attention aux classes natives de PHP (Le global namespace)

Lorsqu'un fichier est déclaré à l'intérieur d'un namespace, PHP cherche par défaut toutes les classes à l'intérieur de ce même espace. Pour appeler une classe native de PHP (comme `Exception`, `DateTime` ou `PDO`), vous devez obligatoirement la précéder d'un `\` pour indiquer à PHP de la chercher dans l'espace global.

```
<?php
namespace App\Services;

class DateManager {
    public function getCurrentDate() {
        // Le '\' initial est requis pour la classe native DateTime
        return new \DateTime();
    }
}
```

5. Lien avec l'Autoloading (PSR-4)

Dans le développement moderne avec PHP, les namespaces ne sont pas choisis au hasard. Ils suivent la norme **PSR-4** prise en charge par le gestionnaire de dépendances **Composer**. Cette

norme impose que la structure des namespaces corresponde exactement à l'arborescence des dossiers sur votre disque dur (ex: `App\Services\Mailer` doit se trouver dans le fichier `src/Services/Mailer.php`), ce qui permet de charger automatiquement les classes sans jamais écrire de `require` ou d'`include`.

Voici trois exercices pratiques de niveaux progressifs pour maîtriser l'utilisation des namespaces en PHP, accompagnés de leurs solutions détaillées.

Exercice 1 : Éviter les collisions de noms (Niveau Facile)

📝 Énoncé

Vous développez une application de commerce en ligne. Vous devez créer deux classes qui portent le même nom : `Order`.

1. Créez un fichier contenant une classe `Order` située dans le namespace `App\Shop`. Cette classe doit posséder une méthode `getTotal()` qui retourne la chaîne `"Total de la commande : 150€"`.
2. Dans le même script (ou un autre fichier), créez une classe `Order` située dans le namespace `App\Admin`. Cette classe doit posséder une méthode `getHistory()` qui retourne la chaîne `"Historique de l'administration"`.
3. Créez un script d'exécution qui instancie ces deux classes et affiche le résultat de leurs méthodes respectives.

🔔 Solution

```
<?php
// Fichier : ShopOrder.php (par exemple)
namespace App\Shop;

class Order {
    public function getTotal() {
        return "Total de la commande : 150€";
    }
}

// Fichier : AdminOrder.php (par exemple)
namespace App\Admin;

class Order {
    public function getHistory() {
        return "Historique de l'administration";
    }
}

// Fichier : index.php (Script d'exécution)
namespace App; // Optionnel, place le script dans un namespace de base
```

```
// Utilisation des chemins absolus (Fully Qualified Names) pour instancier
$shopOrder = new \App\Shop\Order();
$adminOrder = new \App\Admin\Order();

echo $shopOrder->getTotal() . "\n";
echo $adminOrder->getHistory() . "\n";
```

Exercice 2 : Importation et Alias (Niveau Moyen)

📝 Énoncé

Dans cet exercice, vous allez manipuler le mot-clé `use` ainsi que les alias pour simplifier l'écriture de votre code.

1. Vous disposez d'une classe `Logger` dans le namespace `Core\Utils\Debug`.
2. Vous disposez d'une autre classe `Logger` issue d'une bibliothèque externe dans le namespace `Vendor\Log\External`.
3. Écrivez un fichier PHP dans le namespace `App\Controller` qui importe ces deux classes. Comme elles possèdent le même nom, vous devez obligatoirement attribuer l'alias `ExternalLogger` à la deuxième classe.
4. Instanciez les deux classes dans une fonction ou directement dans le script en utilisant leurs noms simplifiés.

🔔 Solution

```
<?php
// Simulations des classes d'origine
namespace Core\Utils\Debug {
    class Logger {}
}

namespace Vendor\Log\External {
    class Logger {}
}

// ---- Fichier de travail ----
namespace App\Controller;

// Importation des namespaces
use Core\Utils\Debug\Logger;
use Vendor\Log\External\Logger as ExternalLogger; // Utilisation d'un alias

// Instanciation simplifiée
$localLogger = new Logger();          // Instancie Core\Utils\Debug\Logger
$extLogger = new ExternalLogger();    // Instancie Vendor\Log\External\Logger

echo "Les deux loggers ont été instanciés avec succès !";
```

Exercice 3 : Le piège du Namespace Global (Niveau Avancé)

📝 Énoncé

Le code ci-dessous génère une erreur fatale PHP lors de son exécution : `Fatal error: Class 'App\Services\DateTime' not found`.

1. Analysez le code et identifiez pourquoi PHP produit cette erreur.
2. Corrigez le code de **deux manières différentes** :
    - **Méthode A :** En modifiant directement la ligne de l'instanciation de l'objet.
    - **Méthode B :** En ajoutant une instruction d'importation au début du fichier.

```
<?php
namespace App\Services;

class DateFormat {
    public function showDate() {
        $date = new DateTime(); // Cette ligne provoque l'erreur
        return $date->format('d/m/Y');
    }
}

$manager = new DateFormat();
echo $manager->showDate();
```

🔔 Solution

Explication de l'erreur :
Puisque le fichier est déclaré sous le namespace `App\Services`, PHP recherche par défaut la classe `DateTime` à l'intérieur de ce même dossier virtuel (`App\Services\DateTime`). Or, `DateTime` est une classe native intégrée à PHP qui réside à la racine du langage (le namespace global).
Correction - Méthode A (Chemin absolu avec l'antislash global) :
On ajoute un `\` devant `DateTime` pour indiquer à PHP de sortir du namespace actuel et de chercher la classe dans l'espace global.

```
<?php
namespace App\Services;

class DateFormat {
    public function showDate() {
        $date = new \DateTime(); // Notez le antislash '\' initial
        return $date->format('d/m/Y');
    }
}
```

Correction - Méthode B (Importation avec `use`) :

On importe la classe globale en haut du fichier. Dès lors, l'appel dans la méthode peut rester tel quel.

```
<?php
namespace App\Services;

use DateTime; // Importation de la classe depuis le namespace global

class DateFormat {
    public function showDate() {
        $date = new DateTime(); // Fonctionne correctement désormais
        return $date->format('d/m/Y');
    }
}
```